cf. Eqs. (1.47)-(1.48).

The PI algorithm is simply the repeated application of nontruncated rollout, and generates a sequence of stable linear policies $\{\mu^k\}$. By replicating our earlier calculations, we see that the policies have the form

$$\mu^k(x) = L_k x, \qquad k = 0, 1, \ldots,$$

where L_k is generated by the iteration

$$L_{k+1} = -\frac{abK_k}{r + b^2 K_k},$$

with K_k given by

$$K_k = \frac{q + rL_k^2}{1 - (a + bL_k)^2},$$

[cf. Eqs. (1.55)-(1.56)].

The corresponding cost function sequence is

$$J_{\mu^k}(x) = K_k x^2;$$

cf. Eq. (1.54). Part of the classical linear quadratic theory is that J_{μ^k} converges to the optimal cost function J^*, while the generated sequence of linear policies $\{\mu^k\}$, where $\mu^k(x) = L_k x$, converges to the optimal policy, assuming that the initial policy is linear and stable. The convergence rate of the sequence $\{K_k\}$ is quadratic, as indicated earlier. This result was proved by Kleinman [Kle68] for the continuous-time multidimensional version of the linear quadratic problem, and it was extended later to more general problems; see the references given in the books [Ber20a] and [Ber22a] (Kleinman gives credit to Bellman and Kalaba [BeK65] for the one-dimensional version of his results).

Truncated Rollout

An m-step truncated rollout scheme with a stable linear base policy $\mu(x) = Lx$, one-step lookahead minimization, and terminal cost approximation $\tilde{J}(x) = \tilde{K}x^2$ is geometrically interpreted as in Fig. 1.5.11. The truncated rollout policy $\tilde{\mu}$ is obtained by starting at $\tilde{K}$, executing m VI steps using μ, followed by a Newton step for solving the Riccati equation.

We mentioned some interesting performance issues in our discussion of truncated rollout in Section 1.1. In particular we noted that:

(a) Lookahead by rollout may be an economic substitute for lookahead by minimization: it may achieve a similar performance for the truncated rollout policy at a much reduced computational cost.

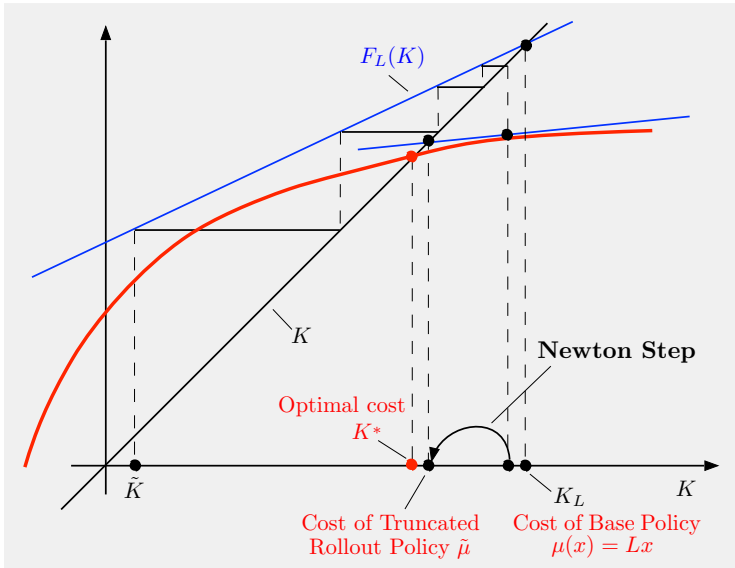

Figure 1.5.11 Illustration of truncated rollout with one-step lookahead minimization, a stable base policy $\mu(x) = Lx$, and terminal cost approximation $\tilde{K}$ for the linear quadratic problem. In this figure the number of rollout steps is $m = 4$.

(b) Lookahead by rollout with a stable policy has a beneficial effect on the stability properties of the lookahead policy.

These statements are difficult to establish analytically in some generality. However, they can be intuitively understood in the context with our one-dimensional linear quadratic problem, using geometrical constructions like the one of Fig. 1.5.11. They are also consistent with the results of computational experimentation. We refer to the monograph [Ber22a] for further discussion.

Double Newton Step - Rollout on Top of Approximation in Value Space

Given a cost function approximation $\tilde{K}$ that defines the policy $\tilde{\mu}(x) = L_{\tilde{K}}x$, it is possible to consider rollout that uses $\tilde{\mu}$ as a base policy. This can be viewed as rollout built on top of approximation in value space, and is referred to as a *double Newton step*; it is a Newton step that starts from the result of the Newton step that starts from $\tilde{K}$ (see Fig. 1.5.12). The double Newton step is much more powerful than approximation in value space with two-step lookahead starting from $\tilde{K}$, which amounts to a value iteration followed by a Newton step. Moreover, the idea of a double Newton step extends to general infinite horizon problems.

Note that it is also possible to consider variants of rollout on top of approximation in value space, such as truncated, simplified, and multi-

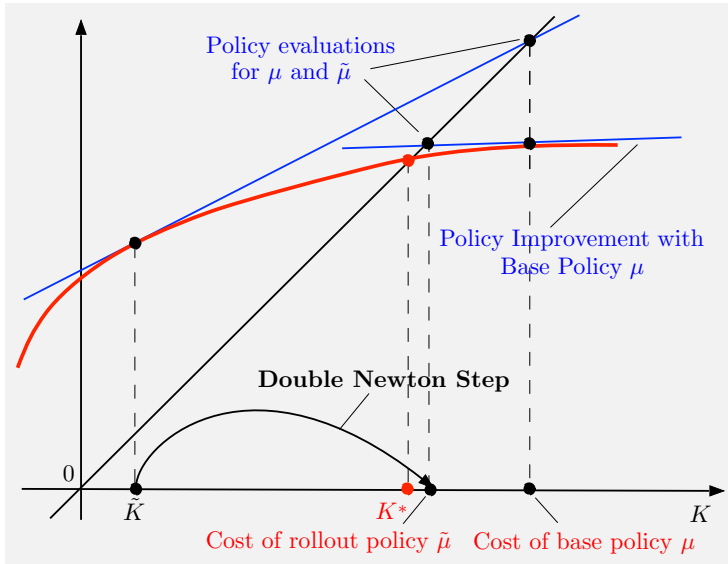

Figure 1.5.12 Illustration of a double Newton step. Here, a base policy μ is obtained by one-step lookahead using cost function approximation $\tilde{J}(x) = \tilde{K}x^2$. The cost function of the corresponding rollout policy $\tilde{\mu}$ is obtained with two successive Newton steps starting from $\tilde{K}$.

step lookahead versions. An important example of the truncated version is the 1996 TD-Gammon architecture [TeG96], where the base policy is obtained through approximation in value space with a terminal cost function approximation that is computed off-line using a neural network.

1.5.3 Local and Global Error Bounds for Approximation in Value Space

In approximation in value space, an important analytical issue is to quantify the level of suboptimality of the one-step or multistep lookahead policy obtained. It is thus important to understand the character of the critical mapping between the approximation error $\tilde{J} - J^*$ and the performance error $J_{\tilde{\mu}} - J^*$, where as earlier, $J_{\tilde{\mu}}$ is the cost function of the lookahead policy $\tilde{\mu}$ and J^* is the optimal cost function.

There is a classical one-step lookahead error bound for the case of an α-discounted problem with finite state space X, which has the form

$$\|J_{\tilde{\mu}} - J^*\| \leq \frac{2\alpha}{1-\alpha}\|\tilde{J} - J^*\|; \tag{1.57}$$

where $\|\cdot\|$ denotes the maximum norm,

$$\|J_{\tilde{\mu}} - J^*\| = \max_{x \in X} \big|J_{\tilde{\mu}}(x) - J^*(x)\big|, \qquad \|\tilde{J} - J^*\| = \max_{x \in X} \big|\tilde{J}(x) - J^*(x)\big|;$$

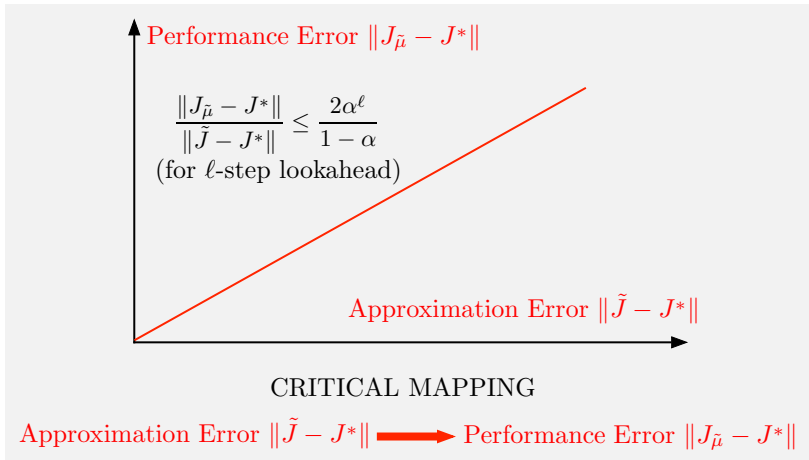

Figure 1.5.13 Illustration of the linear error bound (1.58) for ℓ-step lookahead approximation in value space. For $\ell = 1$, we obtain the one-step bound (1.57).

see e.g., [Ber19a], Prop. 5.1.1. The bound (1.57) predicts a linear relation between the size of the approximation error $\|\tilde{J} - J^*\|$ and the performance error $\|J_{\tilde{\mu}} - J^*\|$. For a generalization, we may view ℓ-step lookahead as one-step lookahead with a terminal cost function $T^{\ell-1}\tilde{J}$, i.e., $\tilde{J}$ transformed by $\ell - 1$ value iterations. We then obtain the ℓ-step bound

$$\|J_{\tilde{\mu}} - J^*\| \leq \frac{2\alpha^\ell}{1-\alpha}\|\tilde{J} - J^*\|. \tag{1.58}$$

The linear bounds (1.57)-(1.58) are illustrated in Fig. 1.5.13, and apply beyond the α-discounted case, to problems where the Bellman equation involves a contraction mapping over a subset of functions; see the RL book [Ber19a], Section 5.9.1, or the abstract DP book [Ber22b], Section 2.2.

Unfortunately, *the linear error bounds are very conservative, and do not reflect practical reality, even qualitatively so.* The main reason is that they are *global* error bounds, i.e., they hold for all $\tilde{J}$, even the worst possible. In practice, $\tilde{J}$ is often chosen sufficiently close to J^*, so that the error $J_{\tilde{\mu}} - J^*$ behaves consistently with the superlinear convergence rate of the Newton step that starts at $\tilde{J}$. In other words, for $\tilde{J}$ relatively close to J^*, we have the *local* estimate

$$\|J_{\tilde{\mu}} - J^*\| = o\big(\|\tilde{J} - J^*\|\big), \tag{1.59}$$

illustrated in Fig. 1.5.14.

A salient characteristic of this superlinear relation is that the performance error rises rapidly outside the region of superlinear convergence of Newton's method. Note that small improvements in the quality of $\tilde{J}$ (e.g., better sampling methods, improved confidence intervals, and the like, without changing the approximation architecture) have *little effect, both inside and outside the region of convergence*.

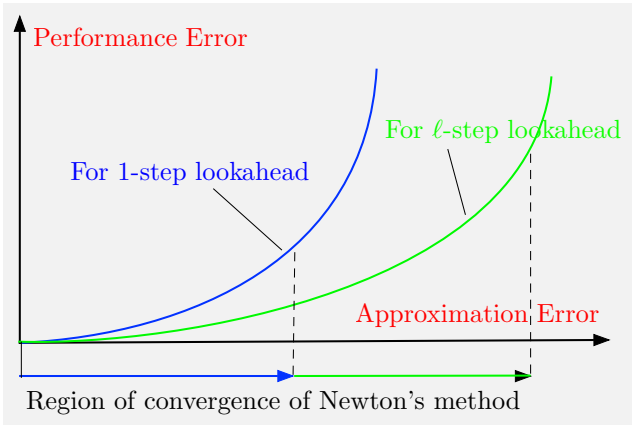

Figure 1.5.14 Schematic illustration of the correct superlinear error bound (1.59) for the case of ℓ-step lookahead approximation in value space scheme. The performance error rises rapidly outside the region of convergence of Newton's method [the illustration in the figure is not realistic; in fact the region of convergence is not bounded as it contains lines of the form γe, where γ is a scalar and e is the unit vector (all components equal to 1)]. Note that this region expands as the size of lookahead ℓ increases. Furthermore, with long enough lookahead ℓ, the ℓ-step lookahead policy $\tilde{\mu}$ can be shown to be *exactly optimal* for many problems of interest; this is a theoretical result, which holds for α-discounted finite-state problems, among others, and has been known since the 60s-70s (Prop. 2.3.1 of [Ber22a] proves a general form of this result that applies beyond discounted problems).

In practical terms, *there is often a huge difference, both quantitative and qualitative, between the linear error bounds (1.57)-(1.58) and the superlinear error bound (1.59)*. Moreover, the linear bounds, despite their popularity in academia, often misdirect academic research and confuse practitioners.† Note that as we have mentioned earlier, the qualitative performance behavior predicted in Fig. 1.5.14 holds very broadly in ap-

† A study by Laidlaw, Russell, and Dragan [LRD23] has assessed the practical performance of popular methods on a set of 155 problems, and found wide disparities relative to theoretical predictions. Quoting from this paper: "we find that prior bounds do not correlate well with when deep RL succeeds vs. fails."

The study goes on to assert the importance of long multistep lookahead (the size of ℓ) in stabilizing the performance of approximation in value space schemes. It also confirms computationally a known theoretical result, namely that with long enough lookahead ℓ, the ℓ-step lookahead policy $\tilde{\mu}$ is exactly optimal (but the required length of ℓ depends on the approximation error $\tilde{J} - J^*$). This fact has been known since the 60s-70s for α-discounted finite-state problems. A generalization of this result is given as Prop. 2.3.1 of the abstract DP book [Ber22b]; see also Section A.4 of the book [Ber22a], which discusses the convergence of Newton's method for systems of equations that involve nondifferentiable mappings (such as the Bellman operator).

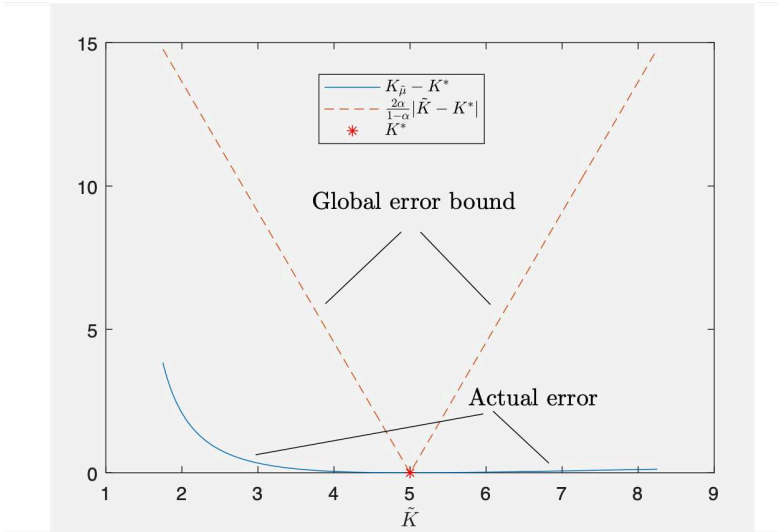

Figure 1.5.15 Illustration of the global error bound for the one-step lookahead error $K_{\tilde{\mu}} - K^*$ as a function of $\tilde{K}$, compared with the true error obtained by one step of Newton's method starting from $\tilde{K}$; cf. Example 1.5.1.

The problem data are $a = 2$, $b = 2$, $q = 1$, and $r = 5$. With these numerical values, we have $K^* = 5$ and the region of stability is (S, ∞) with $S = 1.25$. The modulus of contraction α used in the figure is computed at $\overline{S} = S + 0.5$. Depending on the chosen value of $\overline{S}$, α can be arbitrarily close to 1, but decreases as $\overline{S}$ increases. Note that the error $K_{\tilde{\mu}} - K^*$ is much smaller when $\tilde{K}$ is larger than K^* than when it is lower, because the slope of F diminishes as K increases. This is not reflected by the global error bound.

proximation in value space, because it relies on notions of abstract DP that apply very generally, for arbitrary state spaces, control spaces, and other problem characteristics; see the abstract DP book [Ber22a].

We illustrate the failure of the linear error bound (1.57) to predict the performance of the one-step lookahead policy with an example given in Fig. 1.5.15.

Example 1.5.1 (Global and Actual Error Bounds for a Linear Quadratic Problem)

Consider the one-dimensional linear quadratic problem, involving the system $x_{k+1} = ax_k + bu_k$, and the cost per stage $qx_k^2 + ru_k^2$. We will consider one-step lookahead, and a quadratic cost function approximation

$$\tilde{J}(x) = \tilde{K}x^2,$$

with $\tilde{K}$ within the region of stability, which is some interval of the form (S, ∞). The Riccati operator is

$$F(K) = \frac{a^2 rK}{r + b^2 K} + q,$$

and the one-step lookahead policy $\tilde{\mu}$ has cost function

$$J_{\tilde{\mu}}(x) = K_{\tilde{\mu}} x^2,$$

where $K_{\tilde{\mu}}$ is obtained by applying one step of Newton's method for solving the Riccati equation $K = F(K)$, starting at $K = \tilde{K}$.

Let S be the boundary of the region of stability, i.e., the value of K at which the derivative of F with respect to K is equal to 1:

$$\frac{\partial F(K)}{\partial K}\Big|_{K=S} = 1.$$

Then the Riccati operator F is a contraction within any interval $[\overline{S}, \infty)$ with $\overline{S} > S$, with a contraction modulus α that depends on $\overline{S}$. In particular, α is given by

$$\alpha = \frac{\partial F(K)}{\partial K}\Big|_{K=\overline{S}}$$

and satisfies $0 < \alpha < 1$ because $\overline{S} > S$, and the derivative of F is positive and monotonically decreasing to 0 as K increases to ∞.

The error bound (1.57) can be rederived for the case of quadratic functions and can be rewritten in terms of quadratic cost coefficients as

$$K_{\tilde{\mu}} - K^* \leq \frac{2\alpha}{1-\alpha}|\tilde{K} - K^*|, \tag{1.60}$$

where $K_{\tilde{\mu}}$ is the quadratic cost coefficient of the lookahead policy $\tilde{\mu}$ [and also the result of a Newton step for solving the fixed point Riccati equation $F = F(K)$ starting from $\tilde{K}$]. A plot of $(K_{\tilde{\mu}} - K^*)$ as a function of $\tilde{K}$, compared with the bound on the right side of this equation is shown in Fig. 1.5.15. It can be seen that $(K_{\tilde{\mu}} - K^*)$ exhibits the qualitative behavior of Newton's method, which is very different than the bound (1.60). An interesting fact is that the bound (1.60) depends on α, which in turn depends on how close $\tilde{K}$ is to the boundary S of the region of stability, while the local behavior of Newton's method is independent of S.

1.5.4 How Approximation in Value Space Can Fail and What to Do About It

Let us finally discuss the most common way that approximation in value space can fail. Consider the case where the terminal cost approximation $\tilde{J}$ is obtained through training with data of an approximation architecture such as a neural network (e.g., as in AlphaZero and TD-Gammon). Then there are three components that determine the approximation error $\tilde{J} - J^*$:

(a) The *power of the architecture*, which roughly speaking is a measure of the *representation error*, i.e., the error that would be obtained even if infinite data were available and were used optimally to obtain $\tilde{J}$.

(b) The *error degradation due the limited availability of training data.*

(c) The additional *error degradation due to imperfections in the training methodology*.

Thus *if the architecture is not powerful enough to bring $\tilde{J} - J^*$ within the region of convergence of Newton's method, approximation in value space with one-step lookahead will likely fail, no matter how much data is collected and how effective the associated training method is*.

In this case, there are two potential practical remedies:

(1) Use a more powerful architecture/neural network for representing $\tilde{J}$.

(2) Extend the combined length of the lookahead minimization and truncated rollout in order to bring the effective value of $\tilde{J}$ within the region of convergence of Newton's method.

The first remedy typically requires a deep neural network or transformer, which uses more weights and requires more expensive training (see Chapter 3).† The second remedy requires longer on-line computation and/or simulation, which may run up against some practical real-time implementation constraint. Parallel computation and sophisticated multistep lookahead implementation methods may help to mitigate these requirements (see Chapter 2).

1.6 EXAMPLES, REFORMULATIONS, AND SIMPLIFICATIONS

In this section we provide a few examples that illustrate problem formulation techniques, exact and approximate solution methods, and adaptations of the basic DP algorithm to various contexts. We refer to DP textbooks for extensive additional discussions of modeling and problem formulation techniques (see e.g., the many examples that can be found in the author's DP and RL textbooks [Ber12], [Ber17a], [Ber19a], [Ber20a], as well as in the neuro-dynamic programming book [BeT96]).

An important fact to keep in mind is that there are many ways to model a given practical problem in terms of DP, and that there is no unique choice for state and control variables. This will be brought out by the examples in this section, and is facilitated by the generality of DP: its basic algorithmic principles apply for arbitrary state, control, and disturbance spaces, and system and cost functions.

1.6.1 A Few Words About Modeling

In practice, optimization problems seldom come neatly packaged as mathematical problems that can be solved by DP/RL or some other methodology.

† For a recent example of implementation of a grandmaster-level chess program with *one-step lookahead* and a huge-size (270M parameter) neural network position evaluator, see Ruoss et al. [RDM24].

Generally, a practical problem is a prime candidate for a DP formulation if it involves multiple sequential decisions, which are separated by feedback, i.e., by observations that can be used to enhance the effectiveness of future decisions.

However, there are other types of problems that can be fruitfully formulated by DP. These include the entire class of deterministic problems, where there is no information to be collected: all the information needed in a deterministic problem is either known or can be predicted from the problem data that is available at time 0 (see, e.g., the traveling salesman Example 1.2.3). Moreover, for deterministic problems there is a plethora of non-DP methods, such as linear, nonlinear, and integer programming, random and nonrandom search, discrete optimization heuristics, etc. Still, however, the use of RL methods for deterministic optimization is a major subject in this book, which will be discussed in Chapter 2. We will argue there that rollout and its variations, when suitably applied, can improve substantially on the performance of other heuristic or suboptimal methods, however derived. Moreover, we will see that often for discrete optimization problems the DP sequential structure is introduced artificially, with the aim to facilitate the use of approximate DP/RL methods.

There are also problems that fit quite well into the sequential structure of DP, but can be fruitfully addressed by RL methods that do not have a fundamental connection with DP. An important case in point is *policy gradient* and *policy search* methods, which will be considered somewhat briefly in Chapter 3. Here the policy of the problem is parametrized by a set of parameters, so that the cost of the policy becomes a function of these parameters, and can be optimized by non-DP methods such as gradient or random search-based suboptimal approaches. This generally relates to the approximation in policy space approach, which we have discussed in Section 1.3.3 and we will discuss further in Section 3.4; see also Section 5.7 of the RL book [Ber19a].

A Practical Guide

As a guide for formulating optimal control problems in a manner that is suitable for a DP solution the following two-stage process is suggested:

(a) Identify the controls/decisions u_k and the times k at which these controls are applied. Usually this step is fairly straightforward. However, in some cases there may be some choices to make. For example in deterministic problems, where the objective is to select an optimal sequence of controls $\{u_0, \ldots, u_{N-1}\}$, one may lump multiple controls to be chosen together, e.g., view the pair (u_0, u_1) as a single choice. This is usually not possible in stochastic problems, where distinct decisions are differentiated by the information/feedback available when making them.

(b) Select the states x_k. The basic guideline here is that x_k should encompass *all the information that is relevant for future optimization*, i.e., the information that is known to the controller at time k and can be used with advantage in choosing u_k. In effect, at time k *the state x_k should separate the past from the future*, in the sense that anything that has happened in the past (states, controls, and disturbances from stages prior to stage k) is irrelevant to the choices of future controls as long we know x_k. Sometimes this is described by saying that the state should have a "Markov property" to express an analogy with states of Markov chains, where (by definition) the conditional probability distribution of future states depends on the past history of the chain only through the present state.

The control and state selection may also have to be refined or specialized in order to enhance the application of known results and algorithms. This includes the choice of a finite or an infinite horizon, and the availability of good base policies or heuristics in the context of rollout.

Note that there may be multiple possibilities for selecting the states, because information may be packaged in several different ways that are equally useful from the point of view of control. It may thus be worth considering alternative ways to choose the states; for example try to use states that minimize the dimensionality of the state space. For a trivial example that illustrates the point, if a quantity x_k qualifies as state, then (x_{k-1}, x_k) also qualifies as state, since (x_{k-1}, x_k) contains all the information contained within x_k that can be useful to the controller when selecting u_k. However, using (x_{k-1}, x_k) in place of x_k, gains nothing in terms of optimal cost while complicating the DP algorithm that would have to be executed over a larger space.

The concept of a *sufficient statistic*, which refers to a quantity that summarizes all the essential content of the information available to the controller, may be useful in providing alternative descriptions of the state space. An important paradigm is problems involving *partial* or *imperfect* state information, where x_k evolves over time but is not fully accessible for measurement (for example, x_k may be the position/velocity vector of a moving vehicle, but we may obtain measurements of just the position). If I_k is the collection of all measurements and controls up to time k (the *information vector*), it is correct to use I_k as state in a reformulated DP problem that involves perfect state observation. However, a better alternative may be to use as state the conditional probability distribution

$$P_k(x_k \mid I_k),$$

called *belief state*, which (as it turns out) subsumes all the information that is useful for the purposes of choosing a control. On the other hand, the belief state $P_k(x_k \mid I_k)$ is an infinite-dimensional object, whereas I_k may be finite dimensional, so the best choice may be problem-dependent.